

StateSync – a walkthrough

Written as a rehearsal aid, not a specification. [README.md](#) reports the numbers and [ARCHITECTURE.md](#) records the decisions; this walks the build in the order it happened, so the causality is visible.

Every figure here is read from a committed artifact — [eval/results/](#) , [llm_cache/MANIFEST.json](#) , [docs/seam_bugs.json](#) . Where a figure could not be sourced it is absent rather than estimated.

1. What the system does, and why it needs to

Three systems hold a view of the same transaction. The **payment gateway** is authoritative on whether money moved. The merchant's **order store** is authoritative on whether goods were promised. The merchant's **ledger** is authoritative on what the books say. They are kept in step by *webhooks* — HTTP callbacks the gateway sends when something happens.

Webhooks have three properties that guarantee drift. They deliver **at-least-once**, so duplicates are normal rather than exceptional. They carry **no ordering guarantee**, so a refund notification can arrive before the capture it refunds. And after roughly a day of failed delivery they are **disabled permanently**, with no bulk replay — six hours of a dead endpoint is six hours of events that are simply gone.

The consequence is a fork with no correct side. A merchant who enables webhooks on several events gets duplicate orders. The same merchant, having disabled them, loses paid orders instead. That is not hypothetical: it is [razorpay/razorpay-magento#208](#) , in the gateway's own issue tracker. Either way money moved and the merchant's database does not know it, and today a human notices — usually after a customer complains — and reconciles by hand.

The loop this closes: three sources disagree, find the disagreements, explain them, fix or escalate, report. One loop, six kinds of disagreement.

2. The build, phase by phase

Phase 1 – Foundations

Built: the domain model, a hash-chained append-only audit ledger, a seeded synthetic generator, and the LLM response cache.

File	Job
<code>src/statesync/models/domain.py</code>	<code>Payment</code> , <code>Order</code> , <code>LedgerEntryRecord</code> , <code>Divergence</code> . All money is a strict integer.
<code>src/statesync/models/enums.py</code>	Closed vocabularies – statuses, divergence classes, reason codes.
<code>src/statesync/ledger/canonical.py</code>	The one byte-string that represents an event. Rejects floats anywhere in the body.
<code>src/statesync/ledger/chain.py</code>	<code>sha256(prev_hash + canonical(body))</code> , and a full-chain walk to verify.
<code>src/statesync/ledger/store.py</code>	Postgres persistence, INSERT-only by grant.
<code>src/statesync/generator/synthetic.py</code>	Seeded transactions; nothing reads the wall clock.
<code>src/statesync/llm/cache.py</code>	Responses keyed by <code>sha256(prompt)</code> , on disk.
<code>migrations/001_ledger.sql</code>	The ledger table, plus a role holding <code>SELECT</code> , <code>INSERT</code> only.

Decisions, and what was rejected.

Integer paise everywhere, floats refused at the type boundary. The alternative was `Decimal` , which is accurate but not canonically serialisable in a way that is stable across environments – `0.1 + 0.2` does not render identically everywhere, and one float in an event body makes the hash chain irreproducible between machines. Rejected because the chain has to survive being rebuilt on a stranger's laptop.

Append-only enforced by a Postgres role grant, not by convention. The alternative was a table the application could write freely and a promise not to. Rejected because a hash chain the application can rewrite is not an audit trail – the chain detects tampering, the grant is what makes tampering require a different credential.

The LLM cache built in Phase 1, four phases before the first model call. The alternative was to add it in Phase 5 alongside the provider. Rejected because a

cache retrofitted after the fact records whatever the network happened to return on the last run, rather than the responses the evaluation actually used.

Gate proved: the chain detects a single tampered field and reports its index; it detects a deleted entry; the same seed produces byte-identical output across two separate processes.

Phase 2 – The loop, end to end

Built: deliberate divergence injection, three-way reconciliation, the staleness window, and the first two measurement arms.

File	Job
<code>src/statesync/injector/clean.py</code>	Drops a webhook, duplicates one, abandons after order creation, refunds without booking it.
<code>src/statesync/reconciler/three_way.py</code>	Set operations across the three views; per-payment so throughput is measurable.
<code>src/statesync/reconciler/staleness.py</code>	The 15-minute eligibility window and two-run confirmation.
<code>src/statesync/reconciler/invariants.py</code>	<code>revenue + fee_expense == settled</code> , across the whole batch.
<code>src/statesync/classifier/deterministic.py</code>	The four classes that are pure set operations.
<code>src/statesync/metrics/throughput.py</code>	Integer-microsecond durations; rates derived only at render.
<code>src/statesync/reporting/exceptions_csv.py</code>	The committed exception list.
<code>eval/arms.py</code> , <code>eval/harness.py</code>	The measurement runs and the report.

Decisions, and what was rejected.

Ground truth comes from causing the divergence. The alternative was labelling a dataset after the fact. Rejected because it reintroduces the question "precision against whose labels?" — breaking known-good data on purpose means the correct answer is known exactly, with no labelling step.

Two-run confirmation before any repair. The alternative was acting on first observation. Rejected because a gateway mid-transition does not yet have a consistent answer, and repairing a payment that was merely in flight makes

state worse than leaving it alone. The cost is one cycle of detection latency, stated rather than hidden.

Throughput stored as integer microseconds. The alternative was storing a float rate. Rejected because a float in a stored field eventually reaches an event body, and the chain would stop reproducing.

Gate proved: on 500 records with 124 injected divergences, rules-only detected 124 with 0 false positives.

A correction found here: the generator was emitting authorised-but-uncaptured payments alongside pending orders and calling that clean baseline data. That state is `ORDER_NO_CAPTURE` – the baseline shipped with real divergences the injector had never recorded, so every false-positive figure would have been wrong. Clean batches now contain only captured, refunded and failed payments.

Phase 3 – Repairs and idempotency

Built: one repair per divergence class, the policy gates, and two independent layers of idempotency.

File	Job
<code>src/statesync/executor/repairs.py</code>	The four repairs, each declaring how it stays safe if it runs twice.
<code>src/statesync/executor/runner.py</code>	gate → cap → lease → execute → record.
<code>src/statesync/executor/store.py</code>	Two implementations of one contract: in-memory and Postgres.
<code>src/statesync/policy/idempotency.py</code>	The lease state machine.
<code>src/statesync/policy/gates.py</code>	Class must be auto-repairable; value under threshold.
<code>src/statesync/policy/blast_radius.py</code>	A cap per run; <code>max_repairs=0</code> is a working kill switch.
<code>migrations/002_repairs.sql</code>	<code>repairs.repair_key</code> primary key, <code>orders.payment_id</code> unique.

Decisions, and what was rejected.

The lease is logical, not a Redis TTL. The specification called for `ex=LEASE_SECONDS` on the claim key. That was rejected because it reintroduces the exact bug the lease exists to prevent: when the lease expires Redis **deletes the key**, so the next call sees no claim, takes a fresh one, and a worker that died mid-repair is never surfaced. The claim record instead carries its own timestamp and is retained far longer than the lease it represents, so expiry is computed on read.

Gates run before the lease is claimed. The alternative was claiming first and checking after. Rejected because a repair the policy refuses would leave a claim behind, and a later run with a raised threshold would find a phantom in-progress repair.

Every executor declares an idempotency class. A uniqueness constraint only protects INSERT-shaped repairs; an UPDATE writes no new row, so a row-count check stays green while the value goes wrong. The store therefore exposes no increment method at all – a relative update like `stock = stock + n` cannot be written by accident.

Gate proved: running N repairs M times produces exactly `len(unique_keys)` writes. Mutation testing showed each layer independently catches a double repair: removing the Redis lease alone does *not* cause one, because the constraint catches every duplicate – so the lease contributes avoided work and concurrency behaviour, not correctness.

Phase 4 – Hard cases and escalation

Built: fifteen ambiguous cases, the deterministic verifier, and the escalation packet.

File	Job
<code>src/statesync/injector/hard_cases.py</code>	The fifteen, plus a late-arriving webhook and an unpriceable instrument.
<code>src/statesync/classifier/verifier.py</code>	Accepts a candidate only if it reconciles exactly, cites real artifacts, and declares consistent rates.
<code>src/statesync/classifier/provider.py</code>	The seam a proposer plugs into, and the generation bounds.
<code>src/statesync/classifier/escalation.py</code>	The packet, and the templated action.

Decisions, and what was rejected.

The provider is an interface, filled first by fixtures. The alternative was building the verifier against a live model in Phase 5. Rejected because it would mean the verifier's first exposure to model output was also its first test – inverting the dependency let the verifier, the packet and the two refusal cases be proven against hand-written candidates, including a set shaped exactly like what a hallucinating provider emits.

Every hypothesis is attached, rejected ones included. The alternative was attaching only what verified. Rejected because an escalation reading "ambiguous" hands the human back the same under-determined problem the system just declined; seven candidates off by consistent paise tells an ops person something a bare verdict never does.

The system may report "no generated hypothesis verified" and never "no explanation exists." The second is a claim the architecture cannot support. A list of forbidden phrases and a test over every reason code enforce it.

Gate proved: case 7 – two legitimate orders, same customer, same amount, seconds apart – is not merged. Case 13 reports two verified candidates with different breakdowns. Case 14 attaches every rejected candidate with the reason it failed.

Phase 5 – The AI layer

Built: fee resolution, a real provider behind the existing seam, the fallback chain, and the chaos suite.

File	Job
<code>src/statesync/classifier/fees.py</code>	Payment object first, then a versioned schedule, then <code>FEE_SCHEDULE_UNKNOWN</code> .
<code>src/statesync/classifier/llm_provider.py</code>	Cache-first; counts calls and network calls separately.
<code>src/statesync/llm/client.py</code>	The fallback chain, and a hard offline switch.
<code>src/statesync/llm/providers.py</code>	OpenAI-compatible transport over stdlib <code>urllib</code> , with backoff.
<code>src/statesync/classifier/settlement.py</code>	<code>SETTLEMENT_GAP</code> , reported as not implemented.
<code>config/fee_schedule.yaml</code>	Rates by instrument, versioned with an effective date.

Decisions, and what was rejected.

A rate is never guessed. The alternative was defaulting to a plausible MDR when fee data is missing. Rejected because a guessed fee manufactures a residual that never existed, handing the model a fabricated problem to solve.

A schedule-derived fee may not assert a discrepancy on its own. The schedule is for validation, not a record of what was charged. Using it to flag books that recorded no fee at all marks every unbooked-fee payment as a divergence — guessing by the back door.

Candidate sets are pooled, never substituted. Discovered by regression, see §3.

Provider transport is stdlib `urllib`, not an HTTP dependency. The demo never makes a network call, so a dependency used only by the cache-warming path would be paid for on every clean install.

Gate proved: the propose-verify path resolves entirely from the committed cache with no client configured. Cold generation cost, from `llm_cache/MANIFEST.json` : **8 provider calls, 1493 ms mean, `openai/gpt-oss-120b`** , measured 2026-08-30T07:55:48Z.

Phase 6 – One screen

Built: escalation packet persistence, a read-only API, and a single screen.

File	Job
<code>src/statesync/api/serve_r.py</code>	Two endpoints over committed JSON. Standard library only.
<code>frontend/src/App.tsx</code>	The queue and its filters.
<code>frontend/src/PacketView.tsx</code>	The packet: case, known components, residual, every candidate, outcome.
<code>frontend/src/money.ts</code>	Paise to rupees, in one place.

Decisions, and what was rejected.

One screen, not four. The runs list and metrics tables are read faster in the README, and a green "chain verified" badge is less convincing than a process exiting non-zero — a badge is easy to fake. The escalation packet is the only view here that is genuinely hard to read as text.

No web framework. Two read-only endpoints over static JSON did not justify a framework and its dependency tree on a project whose primary demo is a terminal.

Nothing on the screen writes. No approve button, no run trigger, no dismiss. A UI that can move money can move money by accident, and an escalation should leave the queue because the underlying disagreement was fixed.

Gate proved: the screen renders stored values rather than recomputing them. Two tests exist only to demonstrate that: an arithmetic string of `STORED-NOT-COMPUTED` renders verbatim, and a candidate flagged as matching still reads "matches" when its sum is wrong.

Not built: browser-level end-to-end tests. Playwright could not launch a browser in the build environment, and shipping tests nobody has watched pass would contradict the standard the rest of the project is held to. The two ops journeys run at component level against the real component tree with the network stubbed; layout and browser behaviour are asserted nowhere.

Phase 7 – Hardening and reproducibility

Built: fail-closed enforcement, the templated README, and the reproducibility script.

File	Job
<code>eval/readme.py</code>	Generates <code>README.md</code> ; the template contains no digits.
<code>scripts/reproduce.sh</code>	Clones the committed state and runs setup and verify with no API key.
<code>scripts/demo.sh</code>	The six-beat recording sequence.
<code>scripts/check_no_secrets.sh</code>	Pre-commit hook: blocks a filled <code>.env.example</code> or a staged key.
<code>scripts/stop.sh</code>	Stops this project's containers and nothing else.
<code>docs/seam_bugs.json</code>	The canonical list of same-shape bugs.

Decisions, and what was rejected.

Every README figure is templated from measured output. The alternative was writing the numbers by hand and updating them. Rejected because a hand-typed figure eventually disagrees with the run that produced it – a test fails the build if the file drifts from what regeneration produces.

The test count and wall-clock throughput are excluded from that staleness check. Guarding them means the build goes red for a reason unrelated to correctness, at the worst possible moment. The figures that are *findings* are compared exactly.

Wall-clock throughput lives in a dated snapshot, outside the arm results.

Otherwise `make eval` twice produces different files and the documented determinism check fails.

Gate proved: a clone of the committed state runs `make setup && make verify` green with every provider key unset.

3. Where the design was wrong

Six reversals. Each was believed, then disproved by something specific.

Believed	True	How it surfaced
A Redis TTL on the claim key implements the lease	The TTL deletes the key at expiry, so the stuck-repair branch is unreachable and a crashed worker is never surfaced	Writing the test for the expired-lease state and finding it could not be reached
Failing closed on a broken chain was implemented	<code>Ledger.verify()</code> was correct, thoroughly tested, and called by nothing — a tampered chain would be detected and then ignored while repairs continued	Auditing Phase 7's exit criteria and grepping for a <code>ChainIntegrityError</code> that did not exist
The ledger should book net settled revenue	Net booking understates output GST liability; a merchant owes tax on gross sale value and claims credit on the fee's tax separately	Raised in review; gross booking also gives the residual a name — the gap between the fee booked and the fee charged
The model arm should replace the deterministic candidate set	Substitution destroyed a correct ambiguity: on hard case 13 the model found neither valid decomposition and reported "nothing verified", turning a case the system knew it could not resolve into one it wrongly believed it had	Comparing per-case reason codes between arms after a review asked which case had changed
A rate inside its permitted range is sufficient	A candidate could claim "MDR at 2%" for an amount nothing like 2% of the payment — sum exact, artifact real, rate plausible, explanation fiction	A real model produced exactly that, and the verifier accepted it

Believed	True	How it surfaced
The generator's clean baseline contained no divergences	Authorised payments with pending orders <i>are</i> <code>ORDER_NO_CAPTURE</code> ; the baseline shipped with divergences the injector never recorded	The first false-positive assertion failed on data that was supposed to be clean

4. 8 bugs, one shape

A value computed correctly in one place, then re-derived, misrouted, or ignored somewhere else. Every component test passed. The failure was always between correct components.

Where	What happened
escalation packet -> exceptions.csv	The packet was built and was correct; the CSV re-derived the reason code from a classifier default, so case 13 shipped mislabelled.
hard-case packets vs the eval	base_paise was passed on one path and omitted on the other, so the same packet was checked two different ways depending on who built it.
arm 3's provider	The model provider was constructed, then the hard-case fixture provider was consumed instead – arm 3 read canned answers and reported zero model calls.
warm_cache	One client was probed for the manifest while a different one served the run, so the recorded chain was never the chain that answered.
provider latency	Retry backoff was measured inside the timed call, so sleep was reported as generation cost – a measurement taken at the wrong boundary.
rejection-rate prose	A sentence said 'two times in three' beside a computed 75%. The figure was restated by hand and drifted.
Ledger.verify()	Correct, thoroughly tested, and called by nothing. A tampered chain would have been detected and then ignored while repairs kept writing.
the test suite and eval/results/packets/	Unqualified run_arm calls wrote into the committed packets directory, so running pytest edited the artifacts it then asserted against. The most serious instance: the evidence was being changed by the thing that checks it, and the reproduction claim rests on those artifacts.

Why component tests could not catch these. Every one of them passed its own tests, every time. The packet was built correctly. The chain verified correctly. The provider was constructed correctly. The latency was timed correctly. No component was wrong – the failure lived in the seam between two correct components, which is precisely where a unit test does not look.

What catches them is a test that asserts agreement *across* outputs rather than correctness within one:

`tests/integration/test_reporting_consistency.py` compares the CSV's reason codes against the reported tallies, the row count against the summary line, detected against the per-class sum, and the rendered report against the result it was built from.

The last instance is the worst. Running the test suite overwrote `eval/results/packets/`, so `pytest` edited the artifacts it then asserted against — the evidence was being changed by the thing that checks it. Given the project's central claim is "clone this and reproduce every number", that is the one bug that could have invalidated the claim itself. A session fixture now fingerprints the committed artifacts before and after every run.

5. What it deliberately does not do

SETTLEMENT_GAP is in the taxonomy and is not detected. The class describes a real failure — a payout that does not equal captures minus refunds minus fees — and it is where the propose-verify architecture generalises. But the sandbox produces no genuine settlement behaviour, so any payout data would be manufactured, and an accuracy figure computed against manufactured data is not a measurement. It reports `not_implemented` with the reason, and is never merged into a headline.

The verifier is sound but not complete. It accepts nothing fabricated: a candidate must reconcile exactly, cite artifacts that exist, and declare rates consistent with their own amounts. It cannot detect an explanation that was never proposed — ambiguity is visible only among generated candidates. That is why candidate sets are pooled rather than replaced: the model can add a resolution, never remove an ambiguity.

No transaction entry, no approve button, no dismiss. Nothing on the screen writes. Repairs are decided by the policy engine during a run, never by a click. An escalation leaves the queue when the next run stops finding the disagreement, so there is no way to clear a row without fixing something.

Not fully autonomous, by design. Anything outside the confidence, value or blast-radius bounds goes to a human. That is a stated boundary rather than a limitation: of 124 injected divergences, 7 reached a person.

Throughput excludes database I/O. The reconciler is measured in-memory. The figure is real for the component it measures, and labelled everywhere it appears.

6. Running it

```
make setup          # deps, containers, migrations
make verify         # lint, types, and every test layer
make eval           # the three arms; regenerates every README figure
make readme         # regenerates README.md from that output
make demo           # the six-beat sequence – no API key needed
make ui             # the packets API and the screen
make reproduce      # clone the committed state and verify it, as a
stranger would
make stop           # this project's containers only
```

`make demo` and `make verify` never reach the network: every model response is cached and committed, and a smoke test asserts that with no client configured.

What the numbers say

On 500 records with 124 injected divergences and 15 hard cases:

- rules-only detected 124 with 0 false positives – the expected floor, since four of six classes are exact set operations
- 7 escalations reached a human
- the model resolved 2 of them; one stayed correctly ambiguous
- the verifier ruled on 16 candidates and rejected 12, across four distinct checks – every guard fired on real model output
- fee-schedule coverage 85.7%,
1 transient divergence filtered by the staleness window